

Progetto TPSIT

Alessandro Sicignano

Giugno 2026

Contents

1	Introduzione	2
1.1	Descrizione del progetto	2
1.2	Risultati ottenuti	2
1.3	Tecnologie utilizzate	2
2	Progettazione	4
2.1	Back End (<i>Model</i>)	4
2.1.1	Descrizione	4
2.1.2	Modello E-R	4
2.1.3	Modello logico	5
2.1.4	Funzioni del DB	6
2.1.5	GDRP	7
2.2	Middleware (<i>Controller</i>)	8
2.2.1	Descrizione	8
2.2.2	URI Esposti	8
2.2.3	UML delle Classi utilizzate	12
2.3	Front End (<i>View</i>)	15
2.3.1	Descrizione	15
2.3.2	Tecnologie utilizzate	15
2.3.3	Mockup della Web-App	17
3	Versioning del Software	20
4	Conclusione	20
4.1	Miglioramenti futuri	20
4.2	Risultati	20

1 Introduzione

1.1 Descrizione del progetto

Progettazione di una Web-App che simuli la prenotazione di un evento legato ad un tipo di punto di interesse. Questo progetto si concentra sulla prenotazione di una stanza d'hotel, le informazioni sui *POI* (Point Of Interest) vengono reperite tramite le API di Open Street Map, un progetto Open Source che consente l'accesso libero ai propri dati.

1.2 Risultati ottenuti

- Implementazione delle principali *feature* di autenticazione, tramite accesso "classico" o con OAuth2.
- Possibilità di interagire con i POI, tramite like e commenti.
- Possibilità di salvare i POI.
- Possibilità di prenotare un POI.
- Visualizzazione del miglior percorso disponibile per il raggiungimento di un POI.

1.3 Tecnologie utilizzate

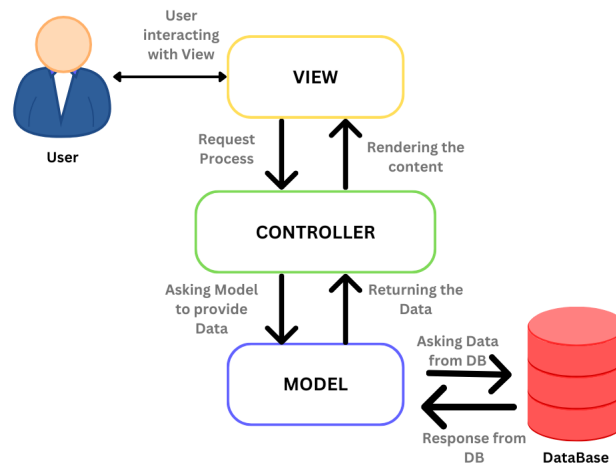


Figure 1: Rappresentazione grafica del modello MVC

Per lo sviluppo del progetto è stato adottato il pattern *MVC* (*Model-View-Controller*) e le principali tecnologie utilizzate sono:

- Linguaggi di programmazione
 - Java
 - TypeScript
 - Overpass QL
 - HTML
 - CSS
- MariaDB
- Tomcat Web Server
- IntelliJ IDEA
- FileZilla (*deploy della Web-App su [labs3](#)*)
- GitLab (*versioning del software nella [repo](#) dedicata*)

2 Progettazione

2.1 Back End (*Model*)

2.1.1 Descrizione

Il Back End è il componente che si occupa dell'interfacciamento tra i *Controller* e i dati nel DB. Il DBMS utilizzato è MariaDB ed il dialogo tra i *Controller* e il DB è stato possibile grazie al Driver "JDBC Type 4" Java MySQL

2.1.2 Modello E-R

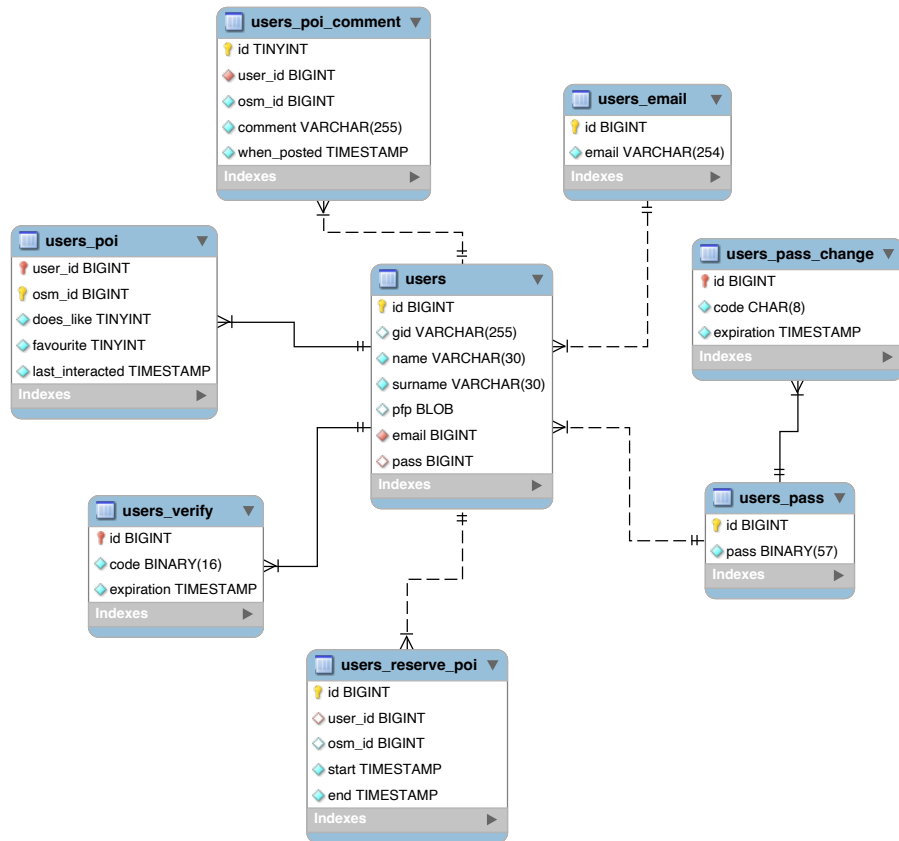


Figure 2: Generato tramite *MySQLWorkbench*

2.1.3 Modello logico

users(id, gid, name, surname, pfp, email*, pass*)

Salva le informazioni pubbliche di un utente. Password e email si trovano su due tabelle diverse.

users_verify(id*, code, expiration)

Salva i codici generati durante la procedura di registrazione degli utenti, i codici hanno una scadenza e sono uno **UUID** univoco per richiesta.

users_pass(id, pass)

Salva il *fingerprint* delle password degli utenti, esso viene generato tramite un algoritmo di *hashing one way*.

users_pass_change(id*, code, expiration)

Salva i codici generati dalle richieste di cambio password degli utenti, i codici hanno una scadenza e non hanno una lunghezza molto elevata per motivi di praticità d'uso da parte degli utenti.

users_email(id, email)

Salva le email degli utenti.

users_poi(user_id*, osm_id, does_like, favourite, last_interacted)

Salva la cronologia dei POI con cui gli utenti hanno interagito e le relazioni esistenti tra di essi. Le relazioni salvate sono se all'utente piace il posto e se l'ha salvato.

users_poi_comment(id, osm_id, comment, when_posted, user_id*)

Salva i commenti lasciati dagli utenti ai POI, gli utenti sono liberi di lasciarne più di uno.

users_reserve_poi(id, osm_id, start, end, user_id*) Salva le prenotazioni degli utenti, essi possono prenotare più di una volta lo stesso POI, con la condizione che prima di effettuare una seconda prenotazione ad un POI già stato prenotato in precedenza, essa debba essere finita o eliminata.

2.1.4 Funzioni del DB

Eventi

Sono stati implementati degli eventi per garantire la pulizia del DB e la non persistenza di dati ormai non più utili.

purge_unused_code

purifica il DB dai codici di cambio password ormai scaduti

purge_non_verified

purifica il DB dai codici di verifica email scaduti ed elimina anche gli account ad essi collegati

Procedure

Sono state implementate varie procedure per permettere interrogazioni efficienti al DB e mitigare attacchi SQL injection.

Procedure principali:

INSERT_NORMAL_USER

Utilizzata per inserire utenti "normali" (non *Google*), viene utilizzata una procedura in modo da garantire che i record contenenti la password e l'email dell'utente siano collegati correttamente ad esso.

VERIFY_USER

Utilizzata per verificare l'email di un utente, la funzione ha la particolarità di ritornare un "codice di stato" che specifica il risultato dell'operazione, (riuscita, codice non trovato, codice trovato ma scaduto)

Trigger

È stato implementato il trigger **check_reservation** per garantire che le prenotazioni inserite rispettino il vincolo di essere "univoche" per periodo di prenotazione.

2.1.5 GDRP

Per conformarsi alle direttive principali europee sulla *privacy* sono state adottate varie tecniche.

Pseudonimizzazione

Tecnica che prevede la separazione logica delle informazioni sensibili dell'utente. In questo progetto è stata adottata durante la definizione delle tabelle *users*, *users_email* e *users_pass*. Per questo motivo la tabella utenti non contiene tutte le informazioni utili all'identificazione di un individuo, ma le informazioni sensibili vengono sostituiti da degli "*alias*", in modo da rendere più difficile e meno immediata la profilazione di un utente.

Minimizzazione

Tecnica che prevede la richiesta del minimo numero di informazioni all'utente

Hashing della password

Tecnica che prevede l'utilizzo di una funzione *hash one way* sulla password in chiaro dell'utente, in questo modo, in caso di attacco da parte di un malintenzionato, anche con tutti i dati presenti nel DB non avrebbe accesso alle password "in chiaro" dell'utente. In questo progetto è stato utilizzato l'algoritmo di hashing *scrypt*, che preso in input una password ed un *salt* generato casualmente genera un *fingerprint* resistente ad attacchi "brute force".

2.2 Middleware (*Controller*)

2.2.1 Descrizione

Il Middleware è il componente che si occupa della comunicazione tra il *Model* e la *View*. La tecnologia principale utilizzata per lo sviluppo di questo componente sono state le **Servlet**. Sono una classe *Java* eseguita da *Tomcat* che agisce da Container di varie Servlet e gestisce le richieste HTTP dei vari Client reindirizzandole al servizio richiesto. Ad una Servlet viene assegnato un URI alla quale essa risponderà, all'interno della classe è possibile specificare i vari tipi di metodi a cui il servizio risponde e in che modo. Il ciclo di vita di una Servlet viene gestito dal Container, essa è un thread il cui scopo è rispondere alla richiesta del Client per poi venir "distrutta". Ogni Servlet implementa un metodo di "Session Filtering" in modo da garantire che un utente non autenticato sia impossibilitato dall'eseguire azioni alla quale non ha accesso.

2.2.2 URI Esposti

/signin

Registrazione di utenti "normali" (non *Google*).

POST

Metodo utilizzato per inizializzare una nuova procedura di registrazione utente, esso non è ancora abilitato ad accedere al proprio profilo, in quanto deve prima verificare la propria email. Questa Servlet si occupa quindi di inviare al DB le informazioni necessarie alla creazione di un account, e di inviare all'utente l'email di verifica.

GET

Metodo utilizzato per finalizzare una procedura di registrazione, prende come parametro un solo valore, ovvero il codice generato dalla richiesta POST (UUID) e richiede al DB la cancellazione del record con il codice appena inviato, in questo modo si evita che l'evento di "pulizia" cancelli il record assieme all'account nel caso in cui il codice risulti scaduto.

/login

Accesso ad un account.

POST

Metodo utilizzato per l'autenticazione di un utente e la creazione di una sessione autenticata, in seguito si viene rimandati alla Servlet */load* che si occupa del caricamento della "pagina principale".

/google

Accesso ad un account *Google*, richiamata automaticamente in seguito ad un accesso valido dal Google button.

POST

Metodo utilizzato per regolare gli accessi con Google, a livello di URI non vi è distinzione tra accesso e registrazione come per gli utenti normali, entrambi vengono gestiti da questa Servlet che si occupa di validare il JWT ricevuto dall'API di Google, e in seguito di capire se si tratta di un account già esistente o se necessita di essere creato. Viene anche saltata la procedura di verifica, in quanto la email viene già “garantita” da Google. In seguito questa Servlet rimanda a */load*.

/load

Caricamento delle informazioni necessarie a *map.jsp*.

GET

Metodo utilizzato per elaborare e inviare alla *View* i dati dell'utente, non vi è alcun parametro, il riconoscimento dell'utente avviene tramite la sessione creata in precedenza da */login* o */google*.

/image

Invio delle foto profilo.

GET

Metodo utilizzato per l'invio della foto profilo di un utente, il suo utilizzo richiede una sessione convalidata. Prende come parametro l'id dell'utente della quale si vuole ricevere la foto profilo, in caso non vi sia alcun parametro si riceve quella del profilo legato alla sessione. Questa Servlet è stata implementata per evitare l'*embedding* dell'immagine all'interno dell'attributo "src" del tag "img" *HTML*.

/logout

Invalida una sessione.

GET

Metodo utilizzato per scollegare la sessione corrente dal proprio id utente, in questo modo il Container può rilasciare tutte le risorse legate ad essa.

/change

Cambio password di un utente

POST

Metodo utilizzato per inizializzare una procedura di cambio password, si occupa di salvare il codice nel DB e di inviarlo via email all'utente.

GET

Metodo utilizzato per finalizzare una procedura di cambio password. Prende come parametro il codice, la nuova password e la email del richiedente.

/getmeta

Invio dei metadati legati ad un POI.

GET

Metodo utilizzato per l'invio dei metadati (numero di like, commenti, e relazioni tra utente e POI) ad una *View* che in seguito si occuperà di elaborarli. È una REST API in quanto si occupa semplicemente di richiedere dei dati al DB e di inviarli in forma di JSON al Client. Prende come parametro l'id del POI e lo collega all'id utente della sessione.

/updatemeta

Aggiornamento delle relazioni tra utenti e POI.

POST

Metodo utilizzato per salvare un nuovo commento nel DB, in seguito risponde al Client con l'id del commento in caso di inserimento avvenuto con successo.

GET

Metodo utilizzato per modificare o creare relazioni (like e/o salvataggi) tra utenti e POI.

DELETE

Metodo utilizzato per eliminare un commento, prende come parametro l'id del commento da eliminare.

/reserve

Creare/Eliminare prenotazioni.

POST

Metodo utilizzato per inserire una prenotazione nel DB, in seguito rimanda ad una pagina che mostra il risultato dell'operazione ed invia un'email con il resoconto di essa all'utente.

GET

Metodo utilizzato per cancellare una prenotazione, prende come parametro l'id della prenotazione da eliminare.

/changeprofile

Modifica delle informazioni degli utenti.

GET

Metodo utilizzato per elaborare i dati necessari alla pagina di modifica, in seguito rimanda i dati a *change_profile.jsp*.

POST

Metodo utilizzato per aggiornare i dati di un utente, prende come parametro i dati che si vogliono modificare, in caso non ne vengano forniti la richiesta non ha alcun effetto.

2.2.3 UML delle Classi utilizzate

Entità del DB trasposte

Classi utilizzate nello scambio di informazioni tra *View* e *Controller*, nel momento dell'invio vengono convertite in JSON.

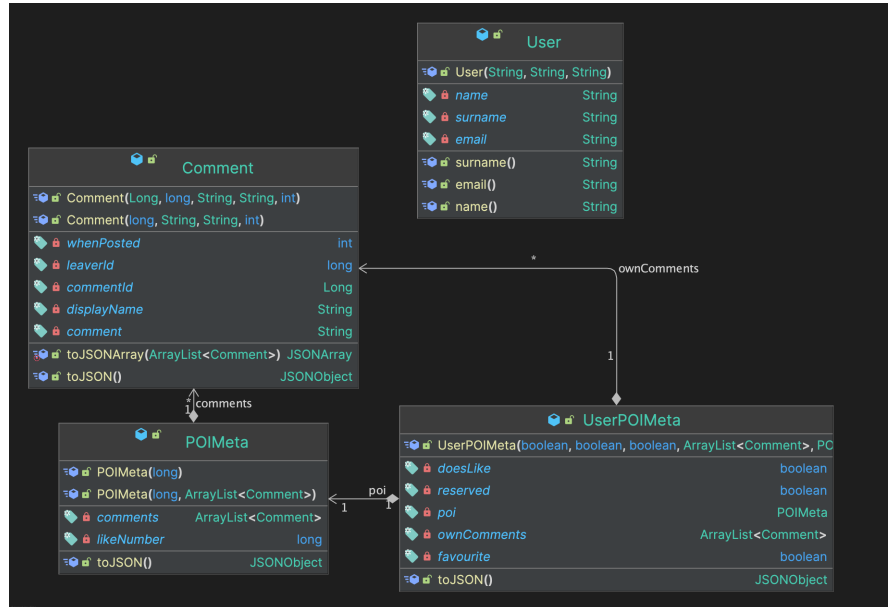


Figure 3: UML delle entità del DB trasposte a classe Java

Classi helper

Classi utilizzate per dividere la logica di alcune operazioni e ridurre la ripetitività del codice. Tra le più importanti ci sono:

- *ServletHelper*, generalizza i controlli sulle sessioni e sui parametri inviati
- *MailHelper*, automatizza il processo di "compiling" di una *jsp* prima del suo invio via email

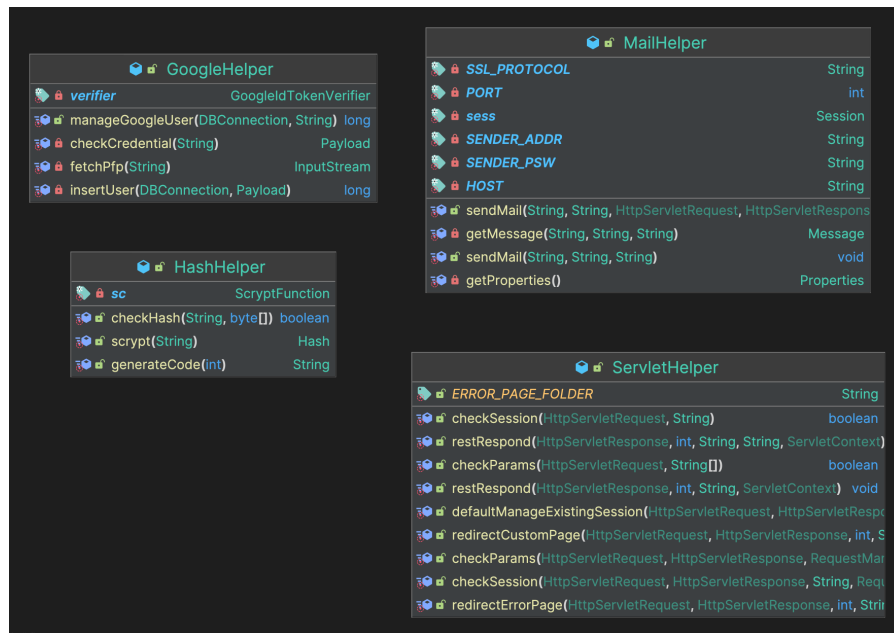


Figure 4: UML delle classi helper

Classi "oggetto"

Classi il cui scopo è simile a quello delle *Helper* ma che prima di poter essere usate vanno istanziate.

La più importante è *DBConnection*, che serve a dividere la logica di interazione con il DB (*Model*) e la chiamata alle sue procedure.

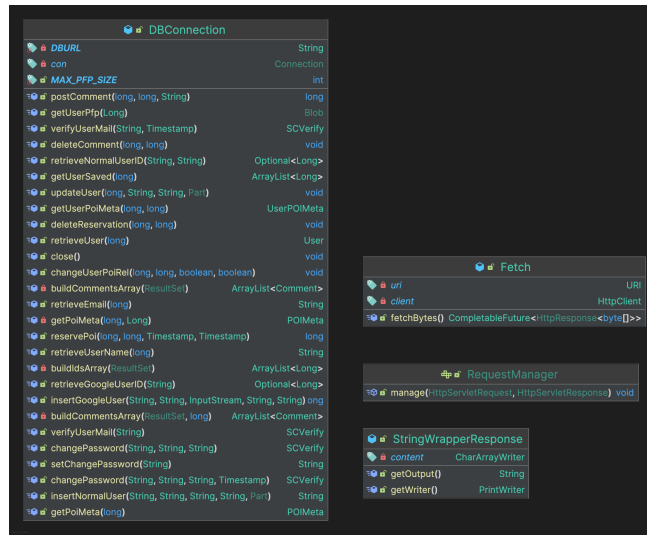


Figure 5: UML delle classi "istanziabili"

2.3 Front End (*View*)

2.3.1 Descrizione

Il Front End è il componente che viene visualizzato dall'utente finale (Client), esso è composto per la maggior parte da JSP (Java-Server Pages) "compilate" in un HTML, CSS, JavaScript. Per "compilazione" si intende il processo che avviene quando un *Controller* fa il "forward" di una richiesta ad una JSP, questa richiesta sarà arricchita di vari attributi impostati dalla Servlet che serviranno al rendering di una risposta al Client. Solo dopo aver "compilato" ovvero processato gli eventuali Scriptlet e aver sostituito i vari *placeholder* con i valori degli attributi, verrà inviata una pagina HTML/CSS di risposta al Client. Ogni JSP implementa un metodo di "Session Filtering" in modo da evitare che un utente già autenticato debba ripetere l'accesso.

2.3.2 Tecnologie utilizzate

Script

Gli script sono un elemento fondamentale del progetto perchè si occupano dei primi controlli Front End, ma soprattutto rendono possibile l'implementazione della mappa. Per "Script" si intende un insieme di due bundle JavaScript compilati da TypeScript tramite Webpack, grazie all'uso di questa tecnica tutte le dipendenze necessarie al corretto funzionamento della Web-App si trovano in un unico file. La mappa è stata implementata tramite la libreria *Open Layers* e tramite i dati reperiti da *OpenStreetMap*, entrambi sono progetti open source. Open Layers fornisce delle classi semplici e intuitive che permettono l'implementazione di mappe dinamiche in una Web-App, mentre OpenStreetMap fornisce liberamente dati geografici di tutto il mondo, e mette anche a disposizione varie API per poter interagire con essi. Alcune di queste API sono state utilizzate nel progetto e sono le seguenti:

- *Nominatim*: permette di ricevere dati precisi su vari POI nel mondo.
 - [/reverse](#), ricava delle informazioni su un POI partendo dalle sue coordinate.
 - [/search](#), permette di cercare informazioni partendo da una stringa di testo.
 - [/lookup](#), ricava informazioni su un POI tramite il suo ID OSM.
- *Overpass*: servizio alla quale è possibile effettuare query strutturate.

Ottimizzazione delle query

Negli Script della mappa si cerca di ottimizzare il più possibile le query, specialmente quelle verso servizi esterni. Per fare ciò vengono applicati vari meccanismi di "caching" che si basano sul fatto che difficilmente un dato geografico è soggetto a cambiamenti. La principale criticità è che questo approccio può compromettere l'aggiornamento dei metadati in tempo reale, ad esempio se un altro utente mette like al POI, la sua interazione non sarà visibile immediatamente agli altri utenti, in caso abbiano il POI nella cache. Un altro meccanismo di ottimizzazione adottato è quello di limitare le query di aggiornamento della relazione tra POI e utente: quando l'utente mette like o salva un POI l'azione non viene immediatamente comunicata al Server. La relazione viene aggiornata solamente in locale dando un'illusione di responsività istantanea all'utente, in realtà la modifica di tale relazione avviene solo dopo che il popup del POI viene chiuso.

2.3.3 Mockup della Web-App

Di seguito una breve "Demo" delle pagine esplorabili nella Web-App con i relativi *redirect*

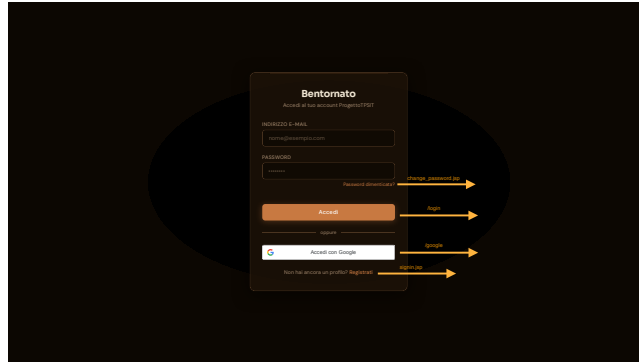


Figure 6: *index.jsp* - pagina di login

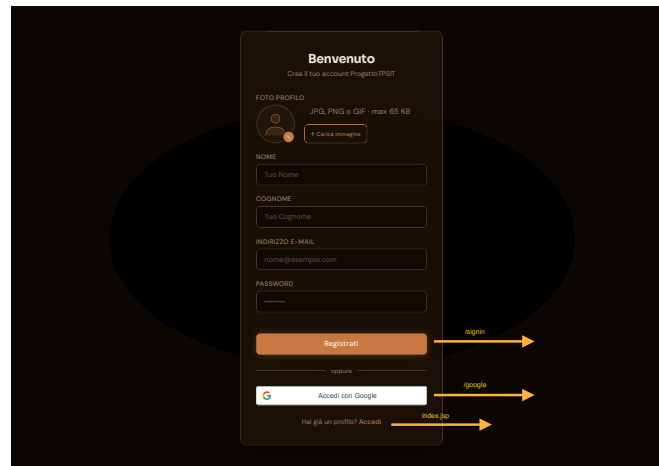


Figure 7: *signin.jsp* - pagina di registrazione

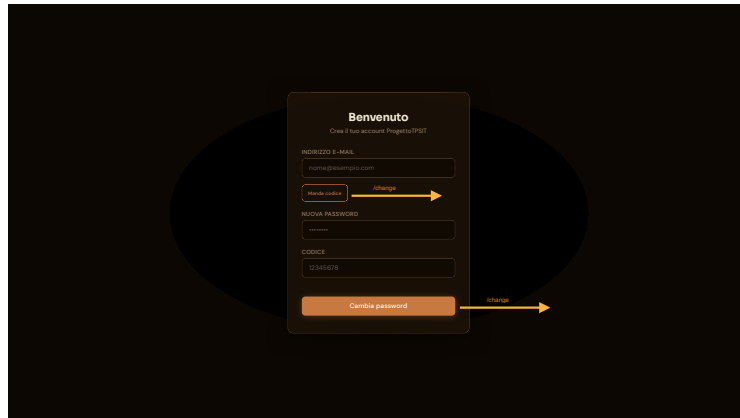


Figure 8: *change_password.jsp* - pagina di recupero password

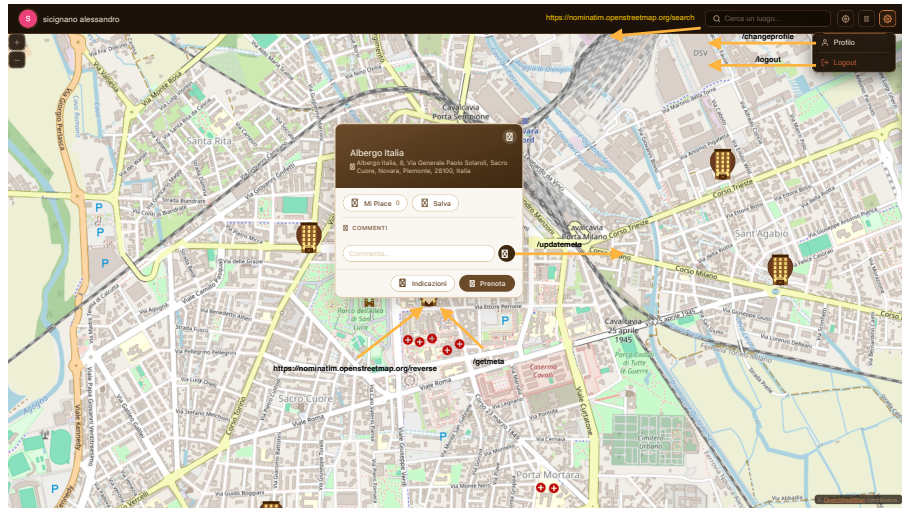


Figure 9: *map.jsp* - Per il rendering dei POI in questa pagina viene effettuata una query a Overpass, vengono richiesti tutti i nodi di tipo hotel all'interno di una bounding box. La bounding box è rappresentata dallo schermo del Client più un buffer che varia in base allo zoom della mappa.

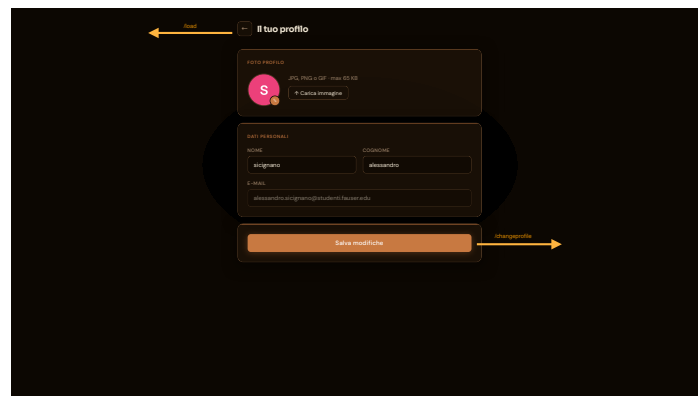


Figure 10: *signin.jsp* - pagina di registrazione

3 Versioning del Software

Il versioning del software è stato eseguito tramite Git ed il salvataggio in cloud tramite GitLab, lo sviluppo si è diviso in vari *branch* locali, per effettuare il testing di nuove funzioni, mentre in remote è stato pushato solamente il branch "dev", che in questo progetto ha agito da "main" del developer dove venivano pushate le funzioni testate e pronte per essere usate.

4 Conclusione

4.1 Miglioramenti futuri

Questo progetto ha ancora molto margine di crescita, alcune implementazioni sono ancora "acerbe" e sono sicuramente migliorabili, alcuni miglioramenti possibili sono:

- Miglioramento dell'interfaccia di prenotazione, appoggiandosi anche ad API esterne.
- Possibilità di filtrare i vari hotel basandosi su informazioni reali.
- Espansione delle possibilità di personalizzazione del proprio profilo.
- Implementazione di un algoritmo che dia indicazioni in tempo reale per il raggiungimento di un luogo
- Aumento della responsività tra le interazioni tra utenti e POI

4.2 Risultati

Questo progetto è stato un ottimo strumento di apprendimento, mi ha introdotto a nuove tecnologie e, anche se in minima parte, anche alle logiche di progettazione adottate nel mondo del lavoro.